



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE

Magíster en Inteligencia Artificial

Evaluación 2

BDD: Lógica y Formas Normales

Curso: INF3831 — Bases de Datos

Profesor: Ariel Leandro Reyes Pardo

Alumno: Armando Arredondo Valle

Preguntas respondidas: **Pregunta 1** y **Pregunta 3**

Mayo 2026

Selección de preguntas

De acuerdo con las instrucciones de la evaluación, se deben responder **dos** de las tres preguntas: la Pregunta 3 es obligatoria, y se puede elegir entre la Pregunta 1 o la Pregunta 2.

Pregunta	Tema	Tipo	Estado
P1	Lógica en la BDD (Stored Procedure + Trigger)	Opcional	Respondida
P2	Transacciones y ACID	Opcional	Omitida
P3	Formas Normales	Obligatoria	Respondida

Se optó por la **Pregunta 1** en lugar de la Pregunta 2 porque permite demostrar el diseño de lógica procedimental en SQL (stored procedure y trigger), aplicando directamente el esquema DCCEcommerce definido en el enunciado.

Esquema de la Base de Datos (DCCEcommerce)

Esquema de la Base de Datos

Categoria(id_cate, nombre_cate, descripcion_cate, activo)

Producto(id_prod, sku, nombre_prod, descripcion_prod, *id_cate*, activo)

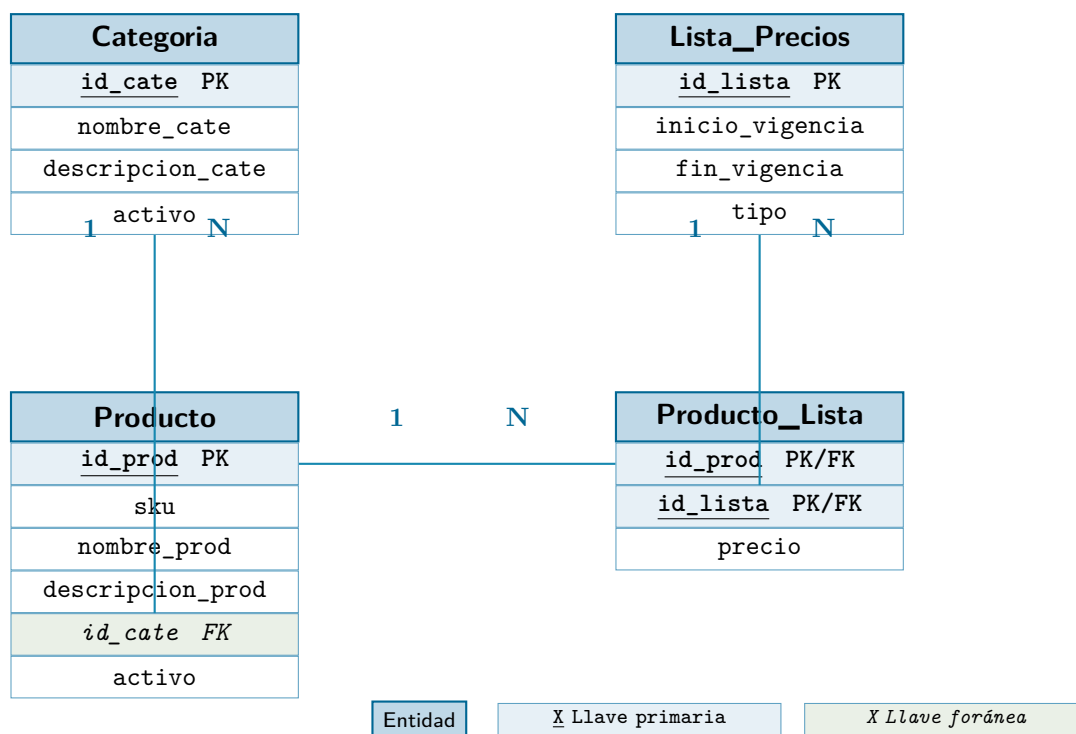
Lista_Precios(id_lista, inicio_vigencia, fin_vigencia, tipo)

Producto_Lista(id_prod, id_lista, precio)

Nota: Llaves primarias subrayadas. Llaves foráneas en *itálica*. Producto.id_cate → Categoria.id_cate; Producto_Lista.id_prod → Producto.id_prod; Producto_Lista.id_lista → Lista_Precios.id_lista.

El campo tipo en Lista_Precios toma valor 0 (lista base) o 1 (lista de oferta). El campo activo en Producto y Categoria es booleano (TRUE/FALSE).

Diagrama Entidad–Relación



Pregunta 1 — Lógica en la BDD

Enunciado P1.1

Cree un **stored procedure** (o función) en PostgreSQL que reciba como parámetros una **fecha** (DATE) y un **identificador de categoría** (id_cate).

El procedimiento debe retornar todos los **productos activos** de esa categoría con su **precio vigente** para la fecha indicada. La lógica de precio es la siguiente:

- Si el producto pertenece a una **lista de oferta** (tipo = 1) vigente en la fecha dada, se retorna ese precio.
- De lo contrario, se retorna el precio de la **lista base** (tipo = 0) vigente en la fecha dada.
- Solo se incluyen productos que tengan al menos un precio vigente (oferta o base).

P1.1 — Función obtener_precios_vigentes(p_fecha, p_id_cate)

Estrategia de solución: se calculan dos CTEs sobre Producto_Lista unida con Lista_Precios: oferta (listas con tipo=1 vigentes en la fecha) y base (listas con tipo=0 vigentes). Luego se usa COALESCE(o.precio, b.precio) para priorizar el precio de oferta sobre el base en el resultado final.

PostgreSQL -- Definición de la función

```

1 CREATE OR REPLACE FUNCTION obtener_precios_vigentes(
2     p_fecha    DATE,
3     p_id_cate  INT
4 )
5 RETURNS TABLE (
6     id_prod    INT,
7     nombre_prod VARCHAR,
8     precio     INT,
9     tipo_precio TEXT
10 )
11 LANGUAGE plpgsql
12 AS $$
13 BEGIN
14     RETURN QUERY
15     WITH oferta AS (
16         -- Productos con lista de OFERTA vigente en la
17         -- fecha dada
18         SELECT pl.id_prod, pl.precio
19         FROM   Producto_Lista pl
20         JOIN   Lista_Precios  lp ON pl.id_lista = lp.
21             id_lista
22         WHERE  lp.tipo = 1
23             AND p_fecha BETWEEN lp.inicio_vigencia AND lp.
24             fin_vigencia
25     ),
26     base AS (
27         -- Precio BASE vigente para cada producto
28         SELECT pl.id_prod, pl.precio
29         FROM   Producto_Lista pl
30         JOIN   Lista_Precios  lp ON pl.id_lista = lp.
31             id_lista
32         WHERE  lp.tipo = 0
33             AND p_fecha BETWEEN lp.inicio_vigencia AND lp.
34             fin_vigencia
35     )
36     SELECT
37         p.id_prod,
38         p.nombre_prod,
39         COALESCE(o.precio, b.precio)           AS
40         precio,
41         CASE WHEN o.precio IS NOT NULL THEN 'oferta'
42             ELSE 'base' END                   AS
43         tipo_precio
44     FROM   Producto p
45     LEFT JOIN oferta o ON p.id_prod = o.id_prod
46     LEFT JOIN base   b ON p.id_prod = b.id_prod
47     WHERE  p.activo   = TRUE
48         AND p.id_cate = p_id_cate
49         AND COALESCE(o.precio, b.precio) IS NOT NULL;
50 END;
51 $$;

```

Invocación de ejemplo

```

1  -- Productos activos de Electronica (id_cate=1) con precio
   al 2026-05-15
2  SELECT * FROM obtener_precios_vigentes('2026-05-15', 1);

```

Resultado de la ejecución (datos de prueba)

id_prod	nombre_prod	precio	tipo_precio
1	Laptop Dell	480 000	oferta
2	Samsung TV 55"	300 000	base
5	Tablet Samsung	150 000	base

Nota: El Laptop Dell obtiene precio de oferta porque existe una `Lista_Precios` con `tipo=1` vigente del 2026-05-01 al 2026-05-31 que le asigna \$480 000 (vs. su precio base de \$520 000). Samsung TV y Tablet Samsung no tienen lista de oferta vigente, por lo que se retorna su precio base.

Decisiones de diseño:

- Se usa `RETURNS TABLE` (función) en lugar de `PROCEDURE` para poder retornar un conjunto de filas directamente con `RETURN QUERY`.
- `COALESCE(o.precio, b.precio)` implementa la prioridad oferta > base en una sola expresión, sin lógica condicional extra.
- El filtro `AND COALESCE(...) IS NOT NULL` excluye productos que no tengan ningún precio vigente (ni oferta ni base) para la fecha solicitada.
- Las fechas de vigencia se evalúan con `BETWEEN`, que es inclusivo en ambos extremos, respetando la semántica del enunciado.

Enunciado P1.2

Cree un **trigger** en PostgreSQL que se dispare **después de insertar** un registro en la tabla `Lista_Precios`.

Al activarse, el trigger debe:

1. Crear la tabla `Productos_Sin_Precio_YYYYMMDD` (donde `YYYYMMDD` es la fecha actual del sistema) si aún no existe.
2. Insertar en esa tabla todos los **productos activos** que **no** aparezcan en ninguna **lista de oferta** (`tipo = 1`).
3. La tabla creada debe contener los atributos: `id_prod`, `sku`, `nombre_prod`, `id_cate`.

P1.2 — Función fn_productos_sin_precio() y Trigger

Estrategia de solución: se crea una función que construye dinámicamente el nombre de la tabla usando TO_CHAR(CURRENT_DATE, 'YYYYMMDD') y ejecuta las sentencias DDL/DML mediante EXECUTE format(...). La función se asocia a Lista_Precios con un trigger AFTER INSERT FOR EACH ROW.

PostgreSQL -- Función del trigger

```

1 CREATE OR REPLACE FUNCTION fn_productos_sin_precio()
2 RETURNS TRIGGER
3 LANGUAGE plpgsql
4 AS $$
5 DECLARE
6     v_tabla TEXT;
7 BEGIN
8     -- Nombre dinamico con fecha de hoy (p.ej.
9     -- Productos_Sin_Precio_20260518)
10    v_tabla := 'Productos_Sin_Precio_'
11              || TO_CHAR(CURRENT_DATE, 'YYYYMMDD');
12
13    -- Crear la tabla si aun no existe (idempotente)
14    EXECUTE format('
15        CREATE TABLE IF NOT EXISTS %I (
16            id_prod      INT,
17            sku           VARCHAR(50),
18            nombre_prod  VARCHAR(100),
19            id_cate      INT
20        )', v_tabla);
21
22    -- Insertar productos activos que NO estan en ninguna
23    -- lista de oferta
24    EXECUTE format('
25        INSERT INTO %I (id_prod, sku, nombre_prod, id_cate)
26        SELECT p.id_prod, p.sku, p.nombre_prod, p.id_cate
27        FROM   Producto p
28        WHERE  p.activo = TRUE
29              AND p.id_prod NOT IN (
30                  SELECT pl.id_prod
31                  FROM   Producto_Lista pl
32                  JOIN   Lista_Precios lp
33                      ON pl.id_lista = lp.id_lista
34                  WHERE  lp.tipo = 1
35              )
36    ', v_tabla);
37
38    RETURN NEW;
39 END;
40 $$;
```

PostgreSQL -- Asociación del trigger

```

1 CREATE TRIGGER trg_lista_precios_insert
2 AFTER INSERT ON Lista_Precios
3 FOR EACH ROW
4 EXECUTE FUNCTION fn_productos_sin_precio();

```

Resultado de la ejecución (datos de prueba — tabla Productos_Sin_Precio_20260518)

Al insertar una nueva `Lista_Precios` con los datos de prueba, la tabla generada contiene los siguientes productos activos sin lista de oferta:

id_prod	sku	nombre_prod	id_cate
2	SKU002	Samsung TV 55"	1
5	SKU005	Tablet Samsung	1
6	SKU006	Jean Levis 501	2

Explicación de resultados:

- *Laptop Dell* (id=1) y *Polera Nike* (id=3) **no aparecen** porque sí tienen una entrada en una lista de oferta (tipo=1).
- *Lámpara* (id=4) no aparece porque `activo = FALSE`.
- Los tres productos incluidos son activos y carecen de precio de oferta.

Decisiones de diseño:

- Se usa `CREATE TABLE IF NOT EXISTS` para tolerar múltiples inserciones en el mismo día sin error (idempotente).
- `format(..., v_tabla)` con el especificador `%I` sanitiza el nombre de tabla dinámico, previniendo inyección SQL.
- El trigger es `AFTER INSERT` (no `BEFORE`) para que el nuevo registro en `Lista_Precios` ya esté confirmado en la tabla al momento de evaluar la condición `lp.tipo = 1`.
- La condición usa `NOT IN (SELECT ...WHERE lp.tipo = 1)` sobre *todas* las listas de oferta históricas, no solo la recién insertada, tal como se indica en el enunciado.

Pregunta 3 — Formas Normales

Enunciado P3

Dada la siguiente relación:

Libros(*ISBN*, *Título*, *AutorID*, *NombreAutor*, *CiudadAutor*)

con las siguientes restricciones semánticas:

- ISBN es la llave primaria (identifica unívocamente cada libro).
- Un libro tiene exactamente un autor.
- Cada autor tiene un único nombre (*NombreAutor*) y vive en una única ciudad (*CiudadAutor*).
- Un autor puede escribir múltiples libros.

P3.1 Identifique todas las dependencias funcionales de la relación.

P3.2 Indique si la relación está en Tercera Forma Normal (3NF). Justifique su respuesta.

P3.3 Descomponga la relación en 3NF (y si es posible, en BCNF). Demuestre que su descomposición preserva dependencias y es sin pérdida.

Instancia de ejemplo para ilustrar la relación original:

ISBN	Título	AutorID	NombreAutor	CiudadAutor
978-0-06-088328-7	Cien años de soledad	1	G. García Márquez	Bogotá
978-0-06-093208-9	El amor en los tiempos del cólera	1	G. García Márquez	Bogotá
978-1-5011-3588-8	La casa de los espíritus	2	Isabel Allende	Lima
978-0-14-243723-0	Ficciones	3	Jorge L. Borges	Buenos Aires

Nótese la **redundancia**: *G. García Márquez* y su ciudad (*Bogotá*) se repiten en cada libro que escribió.

P3.1 — Dependencias Funcionales

A partir de las restricciones semánticas del enunciado se identifican las siguientes dependencias funcionales (DF):

#	Dependencia funcional	Justificación
DF1	$ISBN \rightarrow Titulo$	Cada ISBN identifica un único título.
DF2	$ISBN \rightarrow AutorID$	Cada libro tiene exactamente un autor.
DF3	$AutorID \rightarrow NombreAutor$	Cada autor tiene un único nombre.
DF4	$AutorID \rightarrow CiudadAutor$	Cada autor vive en una única ciudad.
DF5	$ISBN \rightarrow NombreAutor$	Transitiva: $ISBN \xrightarrow{DF2} AutorID \xrightarrow{DF3} NombreAutor$.
DF6	$ISBN \rightarrow CiudadAutor$	Transitiva: $ISBN \xrightarrow{DF2} AutorID \xrightarrow{DF4} CiudadAutor$.

Llaves candidatas: La única llave candidata es $\{ISBN\}$, ya que:

- $ISBN^+ = \{ISBN, Titulo, AutorID, NombreAutor, CiudadAutor\}$ (determina todos los atributos).
- Ningún subconjunto propio de $\{ISBN\}$ lo hace (es atómico).

Atributos primos (parte de alguna llave candidata): solo **ISBN**.

Atributos no-primos: *Titulo, AutorID, NombreAutor, CiudadAutor*.

Nota: No existe la DF $NombreAutor \rightarrow CiudadAutor$ porque dos autores con el mismo nombre podrían vivir en ciudades distintas; la dependencia se ancla en **AutorID** (identificador único de autor).

P3.2 — Análisis de Formas Normales

Se verifica progresivamente desde 1NF hasta 3NF.

Primera Forma Normal (1NF): **Sí cumple.**

Una relación está en 1NF si todos sus atributos contienen valores **atómicos** (no multivaluados ni compuestos) y existe una llave primaria. En *Libros*, cada atributo almacena un valor escalar y ISBN es llave primaria $\Rightarrow$ cumple 1NF.

Segunda Forma Normal (2NF): **Sí cumple.**

Una relación está en 2NF si está en 1NF y no existen **dependencias parciales** (ningún atributo no-primero depende de un subconjunto propio de la llave candidata). Como la llave candidata $\{ISBN\}$ tiene un solo atributo, no puede existir dependencia parcial $\Rightarrow$ la relación cumple trivialmente 2NF.

Tercera Forma Normal (3NF): **No cumple.**

Una relación está en 3NF si, para toda DF no trivial $X \rightarrow A$, se cumple al menos una de las siguientes condiciones:

- (1) X es superclave, **o**
- (2) A es atributo primo.

Las DFs DF3 y DF4 **violan** ambas condiciones:

DF	Dependencia	X es superclave?	A es primo?	Viola 3NF?
DF3	$AutorID \rightarrow NombreAutor$	No	No	Sí
DF4	$AutorID \rightarrow CiudadAutor$	No	No	Sí

Estas son **dependencias transitivas**:

$ISBN \xrightarrow{DF2} AutorID \xrightarrow{DF3} NombreAutor$ $ISBN \xrightarrow{DF2} AutorID \xrightarrow{DF4} CiudadAutor$

$AutorID$ es un atributo no-primo que determina otros atributos no-primos, creando la transitividad que viola 3NF.

Consecuencias prácticas de la violación:

- **Anomalía de inserción:** no se puede registrar un autor sin que exista al menos un libro.
- **Anomalía de actualización:** cambiar la ciudad de un autor obliga a actualizar múltiples filas (una por libro).
- **Anomalía de eliminación:** al borrar el único libro de un autor se pierde la información de ese autor.

P3.3 — Descomposición en 3NF y BCNF

Para eliminar las dependencias transitivas se descompone *Libros* en dos relaciones, separando la información de autores:

Libros(ISBN, *Titulo*, *AutorID*) FK: $AutorID \rightarrow Autores(AutorID)$
Autores(*AutorID*, *NombreAutor*, *CiudadAutor*)

Verificación: ¿cumple 3NF y BCNF?

Tabla	DFs no triviales	3NF?	BCNF?
Libros	$ISBN \rightarrow \{Titulo, AutorID\}$	✓ $ISBN$ es superclave	✓
Autores	$AutorID \rightarrow \{NombreAutor, CiudadAutor\}$	✓ $AutorID$ es superclave	✓

En BCNF toda DF no trivial $X \rightarrow Y$ requiere que X sea superclave. En ambas tablas el único determinante de cada DF es la llave primaria, por lo que satisfacen $BCNF \supset 3NF$.

Propiedad 1 — Lossless-join (sin pérdida de información):

La descomposición es sin pérdida si y solo si el atributo de unión es llave primaria (o superclave) en al menos una de las tablas resultantes. $AutorID$ es llave primaria en

Autores, por lo que:

$$\pi_{ISBN, Titulo, AutorID}(Libros_orig) \bowtie \pi_{AutorID, NombreAutor, CiudadAutor}(Libros_orig) = Libros_orig$$

No se generan tuplas espurias.

Propiedad 2 — Dependency-preserving (preservación de dependencias):

DF original	Dependencia	Preservada en
DF1	$ISBN \rightarrow Titulo$	Libros
DF2	$ISBN \rightarrow AutorID$	Libros
DF3	$AutorID \rightarrow NombreAutor$	Autores
DF4	$AutorID \rightarrow CiudadAutor$	Autores
DF5	$ISBN \rightarrow NombreAutor$	Derivable via JOIN
DF6	$ISBN \rightarrow CiudadAutor$	Derivable via JOIN

Las DFs primitivas (DF1–DF4) están preservadas localmente en las tablas sin necesidad de JOIN. Las DFs transitivas (DF5, DF6) son derivables y no es necesario preservarlas directamente.

Resolución de las anomalías:

- **Inserción:** ahora se puede registrar un autor en **Autores** sin necesidad de que exista un libro.
- **Actualización:** cambiar la ciudad de un autor requiere modificar **una sola fila** en **Autores**.
- **Eliminación:** borrar todos los libros de un autor no elimina su registro en **Autores**.